

Creating an ER Diagram for an E-Commerce Application – Detailed Notes

Introduction

An **Entity Relationship (ER) Diagram** is a visual representation of the database structure that shows:

- Entities (Tables)
- Attributes (Columns)
- Relationships between entities
- Cardinality (One-to-One, One-to-Many, Many-to-Many)
- Primary Keys and Foreign Keys

ER diagrams are usually created **before implementing the database in SQL** because they help developers understand how data is connected.

Why Use an ER Diagram?

ER diagrams help in:

- ✓ Planning the database structure
 - ✓ Identifying relationships between tables
 - ✓ Reducing redundancy
 - ✓ Simplifying SQL table creation
 - ✓ Improving communication among developers
-

Step 1: Identify the Requirements

For an E-Commerce Application, the system should support:

- Customer Registration
 - Product Management
 - Product Categories
 - Shopping Cart
 - Order Placement
 - Payment Processing
 - Product Reviews
-

Step 2: Identify Entities

Entities represent real-world objects.

Entity List

Entity	Description
Customer	Registered users
Category	Product categories
Product	Items available for sale
Cart	Temporary shopping cart
Order	Customer purchase
Order_Item	Products inside orders
Payment	Payment details
Review	Product ratings and comments

Step 3: Define Attributes of Each Entity

1. Customer Entity

Attributes

Customer

customer_id (PK)

name

email

phone

address

created_at

Explanation

Attribute	Description
customer_id	Unique customer identifier
name	Customer name

Attribute	Description
email	Email address
phone	Mobile number
address	Shipping address
created_at	Registration date

2. Category Entity

Category

category_id (PK)

category_name

3. Product Entity

Product

product_id (PK)

product_name

description

price

stock_quantity

category_id (FK)

4. Cart Entity

Cart

cart_id (PK)

customer_id (FK)

product_id (FK)

quantity

5. Order Entity

Order

order_id (PK)

customer_id (FK)

order_date

status

6. Order_Item Entity

Order_Item

order_item_id (PK)

order_id (FK)

product_id (FK)

quantity

price

7. Payment Entity

Payment

payment_id (PK)

order_id (FK)

payment_method

amount

payment_status

payment_date

8. Review Entity

Review

review_id (PK)

customer_id (FK)

product_id (FK)

rating

comment

Step 4: Determine Relationships

Relationships describe how entities interact.

Customer → Orders

Relationship Type

One-to-Many (1:M)

Meaning:

One customer can place many orders.

Example

Customer 1

|

|

├— Order 101

├— Order 102

└— Order 103

Category → Products

Relationship Type

One-to-Many

A category contains multiple products.

Example

Electronics

|

└─ Laptop

└─ Mobile

└─ Camera

Customer → Cart

Relationship Type

One-to-Many

One customer may have many cart items.

Customer

|

└─ Cart Item 1

└─ Cart Item 2

└─ Cart Item 3

Product → Cart

Relationship Type

One-to-Many

A product may appear in many customers' carts.

Order → Order_Items

Relationship Type

One-to-Many

One order contains multiple products.

Order 1001

|

└─ Laptop

└─ Mouse

└─ Keyboard

Product → Order_Items

Relationship Type

One-to-Many

A product can belong to multiple orders.

Order → Payment

Relationship Type

One-to-One

Each order generally has one payment.

Order 1001

|

Payment

Customer → Reviews

Relationship Type

One-to-Many

A customer can write many reviews.

Product → Reviews

Relationship Type

One-to-Many

One product can receive many reviews.

Step 5: Understanding Cardinality

Cardinality specifies how many records are associated.

Relationship	Cardinality
Customer → Orders	1 : M
Category → Products	1 : M
Customer → Cart	1 : M
Product → Cart	1 : M
Order → Order_Items	1 : M
Product → Order_Items	1 : M
Order → Payment	1 : 1
Customer → Reviews	1 : M
Product → Reviews	1 : M

Step 6: Identifying Many-to-Many Relationships

Many-to-Many relationships cannot be implemented directly in relational databases.

They require an intermediate table.

Example 1: Orders and Products

A customer may purchase many products.

A product may be purchased by many customers.

Orders

M

|

Order_Items

|

M

Products

Order_Items acts as a junction table.

Example 2: Customers and Products

Customers can review many products.

Products can receive reviews from many customers.

Customers

M

|

Reviews

|

M

Products

Step 7: Drawing the ER Diagram

Chen Notation Symbols

Symbol	Meaning
Rectangle	Entity
Oval	Attribute

Symbol	Meaning
Diamond	Relationship
Underlined Attribute	Primary Key
Double Oval	Multivalued Attribute
Dashed Oval	Derived Attribute

Crow's Foot Notation

Symbol Meaning

O Zero

< Many

Examples

Customer

|-----<

Orders

Means

One Customer has many Orders.

Step 8: Complete ER Diagram (Text Representation)

```

+-----+
|  CUSTOMER  |
+-----+
| customer_id (PK) |
| name          |
| email         |
| phone        |
| address       |
| created_at    |
+-----+
|
| 1
|
| M
+-----+
|  ORDERS    |
+-----+

```

```

| order_id (PK) |
| customer_id (FK) |
| order_date |
| status |

```

```

+-----+

```

```

|
| 1
|
| M

```

```

+-----+

```

```

| ORDER_ITEMS |

```

```

+-----+

```

```

| order_item_id(PK) |
| order_id (FK) |
| product_id (FK) |
| quantity |
| price |

```

```

+-----+

```

```

|
| M
|
| 1

```

```

+-----+

```

```

| PRODUCTS |

```

```

+-----+

```

```

| product_id (PK) |
| product_name |
| description |
| price |
| stock_quantity |
| category_id (FK) |

```

```

+-----+

```

```

|
| M
|
| 1

```

```

+-----+

```

```

| CATEGORY |

```

```

+-----+

```

```

| category_id (PK) |
| category_name |

```

```

+-----+

```

```

CUSTOMER

```

```

|
| 1

```

|
|M
CART
|
|M
|
|1
PRODUCTS

ORDERS
|
|1
|
|1
PAYMENTS

CUSTOMER
|
|1
|
|M
REVIEWS
|
|M
|
|1
PRODUCTS

Step 9: Tools for Creating ER Diagrams

Tool	Features
MySQL Workbench	Automatic ER diagram generation
Draw.io	Free and easy to use
Lucidchart	Professional diagrams
dbdiagram.io	SQL to ERD conversion
ERDPlus	Educational ER diagram tool
Microsoft Visio	Enterprise-level diagrams

Step 10: Best Practices for Designing ER Diagrams

Use Meaningful Names

✓ Customer

✓ Product

✓ Order_Items

Instead of

✗ Table1

✗ Data2

Identify Primary Keys Clearly

Always underline or mark primary keys.

Example

customer_id (PK)

product_id (PK)

order_id (PK)

Use Foreign Keys Properly

Example

Orders.customer_id

references

Customers.customer_id

Avoid Redundant Data

Store category information only once in the **Category** table and reference it using **category_id**.